

CSC 2114 Artificial Intelligence: Assignment 1

Makerere Campus Route Finder (DFS and BFS)

Group members

#	Full name	Reg. number	#	Full name	Reg. number
1	NOMWESIGWA KEITH	23/U/16071/EVE	4	MUSHEIJA ABRAHAM	23/U/12139/EVE
2	WAMBUI MARIAM	23/U/18494/PS	5	NALUYANGE KEVIN	18/U/23394/EVE
3	LUKAANYA GEORGE	23/U/0696	6	NABIRYE ANITAH	23/U/12826/EVE

1. Problem formulation (Part A)

The state space S is the set of the 18 places in the campus graph: 14 schools and 4 landmarks. A state is simply the place where the visitor is standing.

- A. **Initial state** (an element of S): the source place typed by the user, for example s_0 = School of Law.
- B. **Actions(s)** (a function from S to a set of actions): $\text{Actions}(s) = \{ \text{Go}(n) : n \text{ is a neighbour of } s \}$. The set is read off the graph: an edge (s, n) appears in EDGES , and because connections are two-way, every edge is stored in both directions in an adjacency list. The neighbours are kept in alphabetical order, so the actions are always generated in the same order. For example, $\text{Actions}(\text{School of Law}) = \{ \text{Go}(\text{Freedom Square}), \text{Go}(\text{School of Economics}) \}$.
- C. **Transition model Result(s, a)** (a function from $S \times A$ to S): $\text{Result}(s, \text{Go}(n)) = n$. Walking to a neighbour puts the visitor at that neighbour. The model is deterministic.
- D. **Goal test** (a Boolean test on a state): $\text{IsGoal}(s)$ is true if and only if s = the destination place.
- E. **Path cost** (a function from paths to numbers): the sum of the step costs, where every step cost is $c(s, \text{Go}(n), n) = 1$. The cost of a route is therefore its number of hops. The step cost is 1 because the model deliberately ignores physical distance. We only care about how many connections are walked, and every connection counts the same, so the optimal route is the one with the fewest hops.

Abstraction. The state ignores (1) the actual walking distance and time between places, (2) slope, since Makerere sits on a hill and some walks are steep, and (3) weather and crowds, for example rain or students moving between lectures. Dropping these is valid because they do not change which places are connected, so any route the agent finds is still a real walkable route. It is useful because the state shrinks to a single place name, which keeps the state space at 18 states and makes the search simple and fast.

State versus node. In the tree-like DFS run from School of Law (Section 2.4), the state Central Teaching Facility appears in the node $\text{Law} \rightarrow \text{Freedom Square} \rightarrow \text{CTF}$ (path cost 2) and again in the node $\text{Law} \rightarrow \text{Freedom Square} \rightarrow \text{CTF} \rightarrow \text{CoCIS} \rightarrow \text{CTF}$ (path cost 4). The state is the same, but the parent and path cost are different, so these are two different nodes.

2. Design

2.1 Node, reached and frontier

- **Node.** A **Node** has the four fields from the lecture: **state** (a place name), **parent** (the node it was generated from, or None for the root), **action** (the place walked to) and **path_cost** (hops from the source). The function **solution(node)** follows the **parent** pointers back to the root and reverses the list, which gives the route from source to destination.
- **Reached table.** **reached** is a dictionary that maps a state to the node that first reached it. A child is added to the frontier only if its state is not already in **reached**, and it is recorded in **reached** at the moment it is generated. Because of this, each place enters the frontier at most once. A flag, **use_reached=False** (or **--no-reached** on the command line), turns the table off and gives tree-like search. In that mode a safety cap of 10,000 expansions stops the search.
- **Frontier.** There is one generic **search()** function, and the strategy is passed in as the frontier class. This is the only difference between BFS and DFS:

Algorithm	Frontier	Add children	Remove from
BFS	queue (FIFO), collections.deque	append in alphabetical order	front (popleft)
DFS	stack (LIFO), Python list	push in reverse alphabetical order	back (pop)

DFS pushes the children in reverse alphabetical order so that the alphabetically first neighbour ends up on top of the stack and is therefore explored first.

2.2 Generic search loop (pseudocode)

function SEARCH(problem, Frontier, use_reached):

1. node $\leftarrow$ Node(problem.initial)
2. frontier $\leftarrow$ Frontier containing node

reached $\leftarrow$ {problem.initial: node} (only if use_reached)

expanded $\leftarrow$ 0
3. while frontier is not empty:
4. node $\leftarrow$ frontier.POP() (FIFO for BFS, LIFO for DFS)

if problem.IS-GOAL(node.state): return node, expanded

if expanded = 10,000: return failure (cap reached)
5. expanded $\leftarrow$ expanded + 1
6. for each child in EXPAND(node), in alphabetical order:

if use_reached:

if child.state in reached: skip this child

reached[child.state] $\leftarrow$ child

7. frontier.ADD(remaining children)

return failure (prints "No route found")

The goal test is applied when a node is removed from the frontier (step 4). "Nodes expanded" counts the nodes that are removed, are not the goal, and have their children generated (step 5).

2.3 Interface

The program prompts for the source, destination and algorithm, and prints **Algorithm**, **Route**, **Cost (hops)** and **Nodes expanded** in the required format. Names are matched case-insensitively, after trimming leading and trailing spaces. An unknown place prints a message, lists the 18 valid places and asks again.

An unknown algorithm asks again; neither crashes the program. If the source equals the destination, the route is one place with cost 0.

2.4 Verifying the reached table (B4)

We ran School of Law → Margaret Trowell School with and without **reached**:

Algorithm	reached table	Result	Cost (hops)	Nodes expanded
DFS	on	route found	5	5
DFS	off	stopped at the 10,000 cap, no route returned	–	10,000
BFS	on	route found	4	12
BFS	off	route found	4	51

Why the tree-like DFS fails. Without **reached**, DFS goes Law → Freedom Square → Central Teaching Facility → CoCIS → Central Teaching Facility → CoCIS → ... for ever. Each of these two places is the alphabetically first neighbour of the other ("Central..." comes before "CoCIS...", and "CoCIS" comes before "Freedom..." and "School..."). Because DFS always explores the first child, it bounces round the cycle CTF – CoCIS and never backs up. The graph has cycles (CTF–CoCIS–Statistics and Planning, and the Main Building–Freedom Square–Main Library triangle), so in tree-like search the search tree is infinite even though there are only 18 states. **How **reached** prevents it:** CTF is recorded when it is first generated. When CoCIS is expanded, its child CTF is already in **reached** and is skipped, so DFS has to try Engineering instead. Each state can enter the frontier only once, so the search must finish after at most 18 expansions. Tree-like BFS still succeeds, because it goes level by level, but it does about 4 times the work (51 expansions against 12).

3. Results (Part C)

Abbreviations used in the routes: MB = Main Building, FS = Freedom Square, ML = Main Library, CTF = Central Teaching Facility, ENG = School of Engineering, SBE = School of the Built Environment, MT = Margaret Trowell School of Industrial and Fine Art, LPA = School of Liberal and Performing Arts, SS = School of Social Sciences, EDU = School of Education, ECO = School of Economics, LAW = School of Law, SSP = School of Statistics and Planning, CONAS = College of Natural Sciences, FT = School of Food Technology, Nutrition and Bioengineering, AGR = School of Agricultural Sciences, VET = School of Veterinary Medicine.

#	Source	Destination	Algorithm	Route	Cost (hops)	Nodes expanded
1	ECO	LAW	BFS	ECO → LAW	1	2
2	ECO	LAW	DFS	ECO → LAW	1	16
3	CoCIS	ENG	BFS	CoCIS → ENG	1	2
4	CoCIS	ENG	DFS	CoCIS → ENG	1	16
5	EDU	SS	BFS	EDU → SS	1	3
6	EDU	SS	DFS	EDU → SS	1	17
7	LAW	MT	BFS	LAW → FS → MB → LPA → MT	4	12
8	LAW	MT	DFS	LAW → FS → CTF → CoCIS → ENG → MT	5	5
9	CoCIS	SBE	BFS	CoCIS → ENG → SBE	2	6
10	CoCIS	SBE	DFS	CoCIS → CTF → FS → MB → LPA → MT → SBE	6	6
11	EDU	VET	BFS	EDU → CONAS → FT → AGR → VET	4	13
12	EDU	VET	DFS	EDU → CONAS → FT → AGR → VET	4	4
13	CONAS	ENG	BFS	CONAS → EDU → ML → FS → CTF → CoCIS → ENG	6	16
14	CONAS	ENG	DFS	CONAS → EDU → ML → FS → CTF → CoCIS → ENG	6	6
15	AGR	LPA	BFS	AGR → FT → CONAS → EDU → ML → MB → LPA	6	12
16	AGR	LPA	DFS	AGR → FT → CONAS → EDU → ML → FS → CTF → CoCIS → ENG → MT → LPA	10	10
17	LAW	VET	BFS	LAW → FS → ML → EDU → CONAS → FT → AGR → VET	7	17
18	LAW	VET	DFS	LAW → FS → MB → SS → EDU → CONAS → FT → AGR → VET	8	15

#	Source	Destination	Algorithm	Route	Cost (hops)	Nodes expanded
19	ENG	VET	BFS	ENG → CoCIS → CTF → FS → ML → EDU → CONAS → FT → AGR → VET	9	17
20	ENG	VET	DFS	ENG → CoCIS → CTF → FS → MB → SS → EDU → CONAS → FT → AGR → VET	10	11

Part C conditions. BFS and DFS return different routes on five pairs (rows 7–10 and 15–20). Three pairs are directly connected schools (rows 1–6). Four pairs are at least six hops apart (rows 13–20, from 6 to 9 hops).

4. Analysis

Why BFS always returns a fewest-hops route. In every one of the 10 pairs, the BFS cost is less than or equal to the DFS cost, and on the five pairs where they differ, BFS is strictly cheaper (for example 2 against 6 on rows 9–10). BFS uses a FIFO queue, so it removes all nodes at depth d before any node at depth $d + 1$. The first goal node it removes is therefore at the smallest depth. The condition from the lecture is that **all step costs are equal** (more generally, path cost is a non-decreasing function of depth). Then the shallowest goal is also the cheapest goal. Here every hop costs 1, so depth equals cost, and BFS is optimal. The reached table does not break this, because a FIFO queue generates every state for the first time at its smallest depth.

A pair where DFS returns a longer route: CoCIS → School of the Built Environment (Figure 1). BFS finds CoCIS → ENG → SBE (2 hops). DFS finds CoCIS → CTF → FS → MB → LPA → MT → SBE (6 hops). CoCIS's neighbours in alphabetical order are Central Teaching Facility, School of Engineering, School of Statistics and Planning. DFS generates all three, but the stack puts CTF on top, so it dives into CTF first, then Freedom Square, Main Building, LPA and Margaret Trowell. Engineering, the node one step from the goal, waits near the bottom of the stack the whole time. Margaret Trowell is next to SBE, so SBE is generated, popped and passes the goal test. DFS stops at the **first** goal it removes. It never compares that route with others, so it has no way to know a 2-hop route was waiting in the stack.

[INSERT FIGURE 1 HERE: report/fig1_cocis_to_built_env.png, about 11 cm wide]

Is DFS always cheaper in nodes expanded? No. Over the 10 pairs, BFS expanded 100 nodes in total and DFS 106, so neither is cheaper overall. DFS wins when its alphabetical first choice happens to head towards the goal. EDU → VET (rows 11–12) takes 4 against 13, because College of Natural Sciences is first alphabetically and leads straight down the chain to Veterinary Medicine. It loses badly when the first choice heads away from the goal. ECO → LAW (rows 1–2) are directly connected, and BFS needs only 2 expansions. DFS needs 16, because ECO's first neighbour, Main Library, leads into the rest of the campus, which DFS explores almost completely before it backs up to School of Law. The same happens for CoCIS → ENG (2 against 16) and EDU → SS (3 against 17).

Memory trade-off. BFS has to keep a whole level of the search tree in the frontier, so its memory grows as $O(b^d)$, where b is the branching factor and d is the depth of the goal. In this graph, the frontier grows at the high-degree hubs (Main Building, Freedom Square and Main Library each have 4 neighbours) and at the cycles, which without **reached** put the same places into the frontier again and again. With **reached** the graph is so small that the BFS frontier never holds more than 7 nodes, for any pair of places. But tree-like BFS from ENG to VET (9 hops) expanded 9,043 nodes and had **18,435 nodes in its frontier at once**, while DFS's frontier only grows as $O(b \cdot m)$, one path plus its siblings. Each expansion takes

constant time, but every generated node must be **stored** until it is expanded. On a large graph, BFS therefore runs out of memory long before the running time becomes a problem. That is why the lecture says memory, not time, limits BFS in practice.

Limitation of the equal-cost assumption. Counting hops treats a short walk across the central landmarks the same as a long walk out to the schools at the edge of the campus. A route with fewer hops can therefore be longer in metres. For example, on LAW $\rightarrow$ VET (row 17), BFS's 7-hop route through Main Library might be a longer walk than an 8-hop route if those connections are short. With real walking distances as edge costs, BFS would no longer be optimal, because its optimality depends on equal step costs. We would use **Uniform-Cost Search** (best-first search with $f(n) = g(n)$, the path cost so far), which always expands the cheapest node first and is optimal for positive step costs.

5. Reflection

We would give every connection its real walking distance in metres, taken from a campus map. Hop counts treat a short crossing of Freedom Square the same as a long walk down the hill to the Agricultural and Veterinary schools, so real distances would make the recommended routes match what a visitor would actually walk.